

Teil 2:

Anforderungen und Spezifikation

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Anforderungen.

- Herausforderungen und Methoden im Bereich der Erhebung, Dokumentation und Management von Anforderungen **aufzählen** und **erläutern**



- Detaillierungsgrade von Anforderungen **erläutern**, **auf Eignung prüfen** und bei der Anforderungsanalyse **nutzen**



- Qualitätskriterien auf Anforderungen **anwenden**, um diese zu **bewerten**



- Methoden zum Erheben von Anforderungen sowie deren Stärken und Schwächen **aufzählen**



Logik-basierte Spezifikation.

- Formale Semantik logischer Ausdrücke **benennen** und **erläutern**



- Einfache Eigenschaften des Ein-/Ausgabe-Verhaltens einer Komponente in formaler Logik **ausdrücken**



- Formale Methoden zur Bewertung von Eigenschaften logischer Ausdrücke **benennen** und **anwenden**



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Prüfbare Anforderungen
7. Literatur und Referenzen

Beispiel 1: Instant Messaging Start-Up IMVU

Initiale Annahmen:

- 1 Nutzer von Instant Messaging haben Interesse an 3D Chat
- 2 Nutzer haben starke Bindung an existierende Messaging Clients



Erste Version:

- IMVU baut Add-On für existierende Messaging Systeme
- Erste Version nach 25.000 Zeilen Code

Initialer Misserfolg:

- Add-On wird nicht genutzt
- Zweifel an Validität der Annahmen

Eric Ries, The Lean Start-Up

<https://www.slideshare.net/venturehacks/the-lean-startup-2/13-IMVU>

Beispiel 1: IMVU

Akzeptanztests:

- Tester haben Spaß beim Erstellen von Avataren
- Tester geben an, dass sie Produkt nicht an Freunde empfehlen würden:
"No way" - **unsicher ob es cool ist.**
- **Aber:** Tester wollen im 3D Chat neue Freunde zu finden ohne soziales Risiko durch eigenes Bewerben des Produkts



<https://www.mmogratis.com/imgs/IMVU-gs1.jpg>

Beispiel 1: IMVU

Markt Analyse zeigt:

- Interessierte Kunden nutzen schon bis zu 7(!) Messaging Apps

Revidierte Annahmen:

- ① Nutzer von Instant Messaging haben Interesse an 3D Chat
- ② Nutzer haben **keine** starke Bindung an existierende Messaging Clients

Neue Version:

- Eigenständiges 3D Messaging System und soziales Netzwerk
- Technisch kleine Änderung hat großen Einfluss auf Akzeptanz



An online community where members use 3D avatars to meet new people, chat, create and have fun with their friends



Beispiel 2: Überlingen (2002)

- Mid-Air Kollision von zwei Flugzeugen in der 71 Menschen sterben
- Beide Flugzeuge ausgestattet mit Collision Avoidance System (TCAS)
- TCAS: Onboard System, das bei Gefahr eines sog. Loss of Separation (LOS) Ausweichempfehlungen gibt
- **Ursache (u.A.):**
Fehlende Anforderungen an TCAS
- **Folgen:**
TCAS II hat sog. 'Sense Reversal' Ausweichempfehlungen



- Beide Flugzeuge auf Höhe FL360
- TCAS instruiert TU-154 zu steigen, Boeing 757 zu sinken
- Fluglotse instruiert TU-154 zu sinken und TU-154 folgt Anweisung des Fluglotsen
- Beide Flugzeuge sinken

<http://www.1001crash.com>

<https://www.youtube.com/watch?v=rycXgXPuDWU>

https://en.wikipedia.org/wiki/2002_%C3%9Cberlingen_mid-air_collision

Projekte scheitern wegen fehlender/schlechter Anforderungen

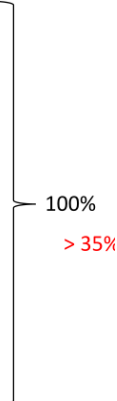
Project Impaired Factors	% of Responses
1. Incomplete Requirements	13.1%
2. Lack of User Involvement	12.4%
3. Lack of Resources	10.6%
4. Unrealistic Expectations	9.9%
5. Lack of Executive Support	9.3%
6. Changing Requirements & Specifications	8.7%
7. Lack of Planning	8.1%
8. Didn't Need It Any Longer	7.5%
9. Lack of IT Management	6.2%
10. Technology Illiteracy	4.3%
Other	9.9%

100%
> 40%

nach <https://www.projectsmart.co.uk/white-papers/chaos-report.pdf>

Erfolgsfaktoren in Projekten

Project Success Factors	% of Responses
1. User Involvement	15.9%
2. Executive Management Support	13.9%
3. Clear Statement of Requirements	13.0%
4. Proper Planning	9.6%
5. Realistic Expectations	8.2%
6. Smaller Project Milestones	7.7%
7. Competent Staff	7.2%
8. Ownership	5.3%
9. Clear Vision & Objectives	2.9%
10. Hard-Working, Focused Staff	2.4%
Other	13.9%



100%
> 35%

nach <https://www.projectsmart.co.uk/white-papers/chaos-report.pdf>

Motivation.

“Es ist wichtig, die Wünsche und Bedürfnisse der Stakeholder zu verstehen, zu dokumentieren sowie die Anforderungen zu spezifizieren und zu managen, um das Risiko zu minimieren, dass das System nicht den Wünschen und Bedürfnissen der Stakeholder entspricht.” [1]

Vorgehen / Methoden.

- Woher wissen wir, was die Software leisten soll?
- Wie beschreiben wir, was die Software leisten soll?

Qualität.

- Haben wir alle (wichtigen) Anforderungen erfasst?
- Haben wir die Anforderungen verständlich dokumentiert?

SWK Anwendungsbeispiel

Remote Home Control

- Geschäftsidee: Kunden können Geräte im Haus kontrollieren und steuern.
- Größeres Software Projekt: Sensoren, Aktuatoren, Server, App, ...

Von der Idee zum System

- [WHY] Was ist der Nutzen?
- [WHAT] Welche Anforderungen muss es erfüllen?



<https://flic.kr/p/oStisy> (CC BY 2.0)

Remote Home Control - Brainstorming

Siehe Chat ...

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Prüfbare Anforderungen
7. Literatur und Referenzen

Anforderungen

Definition (Anforderung)

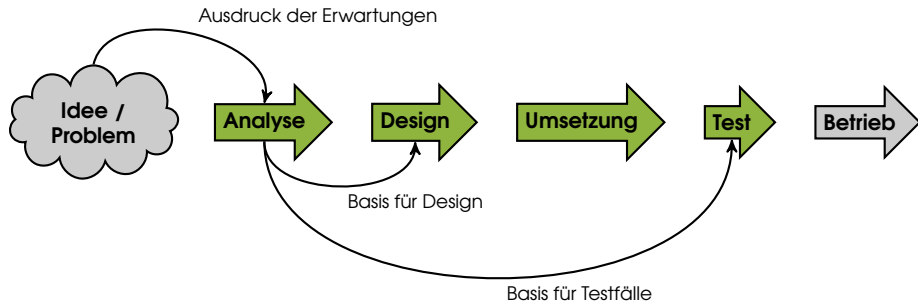
Eine Anforderung ist:

- ① Eine Bedingung oder Fähigkeit, die von einem Benutzer (Person oder System) zur Lösung eines Problems oder zur Erreichung eines Ziels benötigt wird.
- ② Eine Bedingung oder Fähigkeit, die ein System oder Teilsystem erfüllen oder besitzen muss, um einen Vertrag, eine Norm, eine Spezifikation oder andere, formell vorgegebene Dokumente zu erfüllen.
- ③ Eine dokumentierte Repräsentation einer Bedingung oder Eigenschaft gemäß (1) oder (2).

Beispiel:

“Der Benutzer kann das Licht in seinem Haus vom Mobiltelefon aus einschalten und ausschalten.”

Anforderungen im Entwicklungsprozess



Anforderungen als Basis von Verträgen



- **Lastenheft:**
Anforderungen des Auftraggebers
- **Pflichtenheft:**
Beschreibung des zu entwickelnden Systems durch potentiellen Auftragnehmer
 - Basis für Verträge über Software
 - Alle Anforderungen müssen überprüfbar sein
 - Ein objektiver Dritter muss entscheiden können
 - Notfalls Abnahme vor Gericht durchsetzen

Projektbetroffene (Stakeholder)

Definition (Stakeholder)

Projektbetroffene (Stakeholder) sind Einzelpersonen und Organisationen, die aktiv an einem Projekt beteiligt sind oder deren Interessen als Folge der Projektdurchführung oder des Projektabschlusses positiv oder negativ beeinflusst werden können.

Bei Softwareprojekten u. A.:

- die Entwickler
- der Kunde
- die Abteilung, die das Produkt später übernehmen und betreiben wird
- das Management

(vgl. PMBOK)

Systemgrenze und Kontext

Definition (Systemgrenze)

Die Systemgrenze trennt das geplante System von seiner Umgebung. Sie grenzt den im Rahmen des Entwicklungsprozesses gestaltbaren und veränderbaren Teil der Realität von Aspekten in der Umgebung ab, die durch den Entwicklungsprozess nicht verändert werden können.

Definition (Systemkontext)

Der Systemkontext ist der Teil der Umgebung eines Systems, der für die Definition und das Verständnis der Anforderungen des betrachteten Systems relevant ist.

Beispiele für Systemkontext:

- Personen (Stakeholder oder Stakeholdergruppen)
- Systeme im Betrieb (andere technische Systeme oder Hardware)
- Prozesse (z.B. Geschäftsprozesse)
- Dokumente (z.B. Gesetze, Standards)

Requirements Engineering

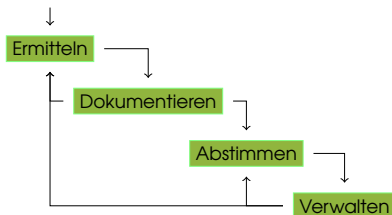
Definition (Requirements Engineering)

Requirements Engineering ist ein systematischer Ansatz zur Spezifikation und zum Management von Anforderungen mit den folgenden Zielen:

- ① Die relevanten Anforderungen zu kennen,
- ② Konsens unter den Stakeholdern über die Anforderungen herzustellen,
- ③ die Anforderungen konform zu vorgegebenen Standards zu dokumentieren und
- ④ die Anforderungen systematisch zu managen.

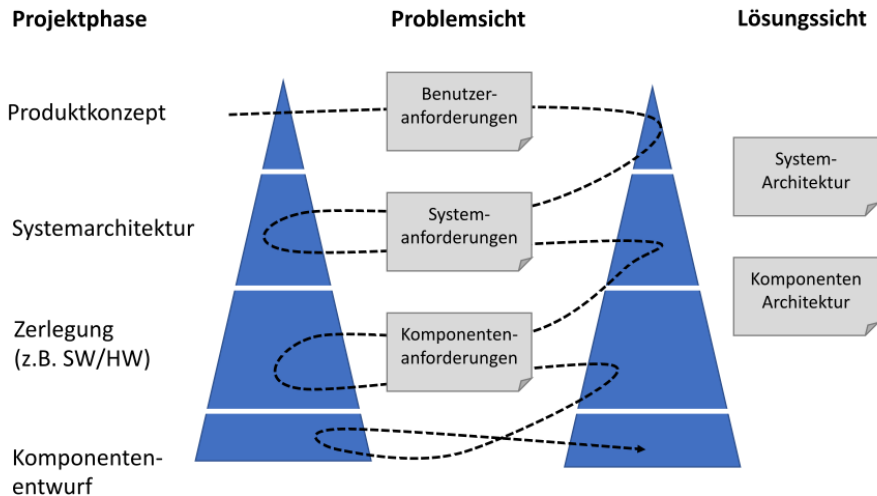
Tätigkeiten:

- Stakeholder und Anforderungen ermitteln
- Anforderungen dokumentieren
- Anforderungen prüfen und abstimmen
- Anforderungen verwalten



Def. nach Pohl, Rupp 2015: Basiswissen Requirements Engineering

Ko-Entwicklung von Architektur und Anforderungen



nach Ebert, 2014: Systematisches Requirements Engineering, 5. Auflage

Arten von Anforderungen

- Es gibt verschiedene Aspekte auf die Anforderungen sich beziehen können bzw. verschiedene Quellen für Anforderungen
- Quellen: Gesetze, Standards, Regeln in der Firma, Stakeholder, etc.
- Aspekte: Funktion, Performanz, Entwicklungsprozess, Werkzeuge, etc.
- Verschiedene Klassifikationsmodelle in der Literatur
 - SWK verwendet Schema nach Pohl, Rupp 2015: Basiswissen Requirements Engineering
 - Sommerville, 2012: Software Engineering benutzt feinere Kategorisierung

Funktionale Anforderungen

Definition (Funktionale Anforderung)

Eine funktionale Anforderung ist eine Anforderung an das Verhalten, das von einer Funktion des Systems bereitgestellt werden soll.

Beispiele:

- *"Der Benutzer kann sich mit seinem Nutzernamen oder seiner E-Mail Adresse anmelden."*
- *"Das System zeigt in der Übersicht zu jedem Gerät im Haus den Status und den Zeitstempel der letzten Interaktion an."*

Qualitätsanforderungen

Definition (Qualitätsanforderung)

Eine Qualitätsanforderung ist eine Anforderung, die sich auf ein Qualitätsmerkmal bezieht, das nicht durch funktionale Anforderungen abgedeckt wird.

Beispiele:

■ Performance:

"Das System muss jede Anfrage des Benutzers innerhalb von 30 Sekunden ausführen (auf System XY)."

"Der Speicherbedarf darf 512MB nicht übersteigen."

■ Zuverlässigkeit:

"Die Verfügbarkeit des Systems muss bei 99.999% liegen."

■ Benutzbarkeit:

„Das System muss die gespeicherten Objekte formatiert ausgeben können nach Schema XXXX.“

Randbedingungen

Definition (Randbedingungen)

Eine Randbedingung ist eine Anforderung, die den Lösungsraum jenseits dessen einschränkt, was notwendig ist, um die funktionalen Anforderungen und die Qualitätsanforderungen zu erfüllen.

Beispiele:

- **Anforderungen an den Entwicklungsprozess:** *„Der Entwickler muss mit dem Kunden monatliche Reviews der zu erstellenden Dokumente durchführen.“*
- **Umgebungsanforderungen:**
„Das System muss mit Java entwickelt werden und muss in der Sun Java VM Version XY laufen.“

Mini-Quiz

Siehe Chat ...

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Prüfbare Anforderungen
7. Literatur und Referenzen



How the customer explained it



How the project leader understood it



How the analyst designed it



How the programmer wrote it



What the beta testers received



How the business consultant described it



How the project was documented



What operations installed



How the customer was billed



How it was supported



What marketing advertised
iSwing



What the customer really needed

Ziel und Herausforderung

Ziel:

- Die relevanten Anforderungen kennen

Herausforderungen:

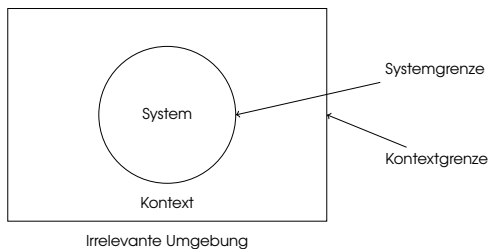
- Stakeholder haben oft implizites Wissen und unbewusste Erwartungen / Vorstellungen
- Kommunikation ist oft unvollständig und ungenau
- Gemeinsames Verständnis muss erst hergestellt werden
- Zugriff auf Stakeholder nicht immer einfach

Vorgehen

- ① Systemgrenze und Kontext ermitteln
- ② Relevante Datenquellen finden (bestehende Software, Gesetze, Normen)
- ③ Angemessene Methoden zum Erheben von Anforderungen auswählen
- ④ Anforderungen von/mit Stakeholdern erheben

Systemgrenze und Kontext identifizieren

Grenzen initial unscharf. RE schärft und verschiebt Grenzen.



- Wo kommen später Daten her? Wo fließen Daten hin?
- Wo werden Schnittstellen benötigt?
- Welche Systeme / Personen interagieren mit dem System?
- Was kann im Entwicklungsprozess verändert werden, was nicht?

Methoden und Hilfsmittel

Methoden:

- Brainstorming
- Beobachtung
- Recherche
 - Marktanalyse
 - Dokumentenanalyse: Normen / Gesetze
 - Systemanalyse
- Befragung
 - Fragebögen
 - Interviews

Hilfsmittel:

- Modelle
- Mind-Maps
- Value Proposition Canvas
- SWOT Analyse

Weitere Methoden und detaillierte Darstellung in der Literatur

Beobachtung und Brainstorming

Beobachtung

Stakeholder werden bei der Arbeit vom Requirements Engineer beobachtet

- **Nutzen:** Unbewusste Anforderungen (z.B. aus Arbeitsabläufen) können gefunden werden
- **Kosten:** Kein Eingriff in die Arbeitszeit der Stakeholder

Brainstorming

Sammeln von Ideen in einer Gruppe in vorgegebener Zeit

- **Nutzen:** Teilnehmer können Ideen anderer aufgreifen und weiterentwickeln
- **Kosten:** Mehrere Stakeholder müssen einen gemeinsamen Termin und Moderator finden
- **Ungeignet:** bei schwieriger Gruppendynamik (z.B. niemand widerspricht dem Chef)

Interview

Gespräche mit Stakeholdern auf Basis von Fragenkatalog / Gesprächsleitfaden. Requirements Engineer wertet Auszeichnungen aus

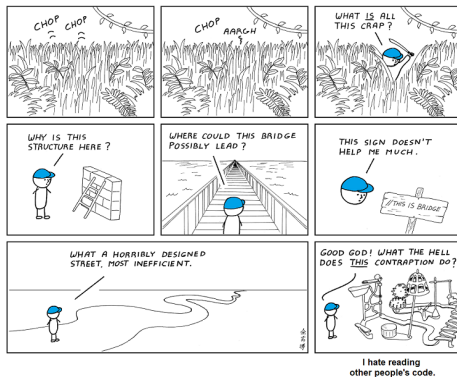
- **Nutzen:** Explizite Anforderungen können in Detail abgefragt werden. Es kann nachgefragt werden.
- **Kosten:** Einzelne Termine mit Stakeholdern Vorbereitung und Auswertung
- **Risiko:** Interviewer lenkt Gespräch zu stark

Fragebogen

Stakeholder werden mit Hilfe von Fragebögen befragt

- **Nutzen:** Explizite Anforderungen können von vielen Stakeholdern abgefragt werden
- **Kosten:** Design eines Fragebogens. Zeit der Stakeholder zum Ausfüllen
- **Schwierigkeit:** Erstellen eines aussagekräftigen Fragebogens ist sehr aufwändig und erfordert fundiertes Wissen

Systemanalyse



- Bestehende Systeme / Kontext können Quelle für Anforderungen sein
- Gefahr: man beschränkt Anforderungen zu sehr auf das, was schon vorhanden ist
- Beispiele: Schnittstellen, Struktur von Daten, Architektur

Hilfsmittel: Modelle

Modelle machen Konzepte erfahrbar und können unbewusste Erwartungen bewusst machen.

Wireframe:

- zeigt Struktur des Produktes

Mock-Up:

- Statisches Modell des Produktes

Prototyp:

- Interaktives Modell des Produktes

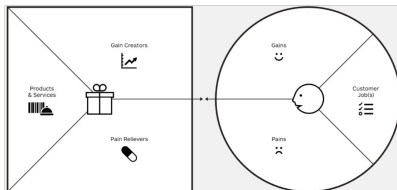


Prototyp nach Byers et al., 2015

Bildquelle: <https://www.sitepoint.com/how-writing-our-book-taught-us-about-killer-prototyping/>

Hilfsmittel: Marktanalyse

Value Proposition Canvas



Analysiert, wie genau ein Produkt Bedürfnisse potentieller Kunden anspricht

- Gain: Nutzen für den Kunden
- Pain: "Schmerzen" des Kunden

SWOT Analyse

Interne Perspektive	Stärken (Strengths)	Schwächen (Weaknesses)
	Chancen (Opportunities)	Risiken (Threats)

Strukturierte Analyse von Unternehmen und Umfeld (z.B. bzgl. des Potentials einer Geschäftsidee)

- Intern: Stärken vs. Schwächen
- Extern: Chancen vs. Risiken

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Prüfbare Anforderungen
7. Literatur und Referenzen

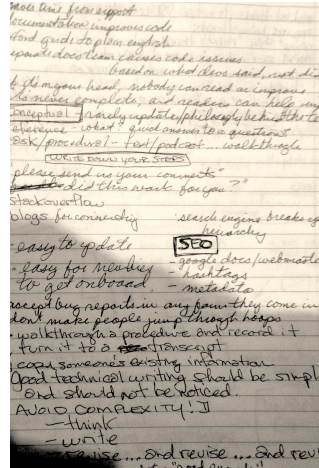
Ziel und Herausforderung

Ziel:

- Die relevanten Anforderungen verständlich und einheitlich dokumentieren

Ansätze:

- Sprache
 - Kann von allen Stakeholdern verstanden und genutzt werden
 - Oft unpräzise
- Modelle
 - Formale Syntax (und ggf. eindeutige Semantik)
 - Modelle abstrahieren



<https://flic.kr/p/dZ37gj>

Dokumentationsformen

Notation	Eigenschaften
Sätze in natürlicher Sprache	Nummerierte Sätze. Jeder Satz sollte eine Anforderung ausdrücken
Strukturierte natürliche Sprache (z.B. Standardformular oder Satz Schablonen)	Anforderungen sind in natürlicher Sprache verfasst. Sätze werden nach Template gebildet. Jeder Satzteil beschreibt einen Aspekt der Anforderung.
Formale Sprache	Es wird eine formale Sprache verwendet (ähnlich einer Programmiersprache). Beschrieben wird erwartetes Verhalten. (Wird heute selten benutzt)
Grafische Notation	Grafische Modelle, ergänzt um Text Annotationen, beschreiben die funktionalen Anforderungen an das System (z.B. UML Anforderungs- und Aktivitätsdiagramme)
Mathematische Spezifikation	Spezifikation basiert auf mathematischem Formalismus (Automaten oder Mengen). Eindeutig aber oft nicht nutzbar, da Kunden diese nicht verstehen.

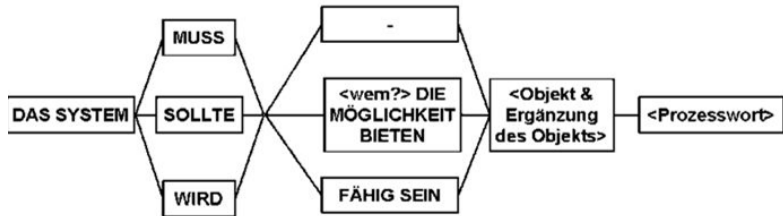
Detailliertere Darstellung: Sommerville, 2012

Natürliche Sprache

„Um Abhängigkeiten zwischen verschiedenen Requests des Fachmoduls zu vermeiden, darf das Fachmodul in der Kommunikation zum Intermediär kein Pipelining verwenden. Parallelität von Requests vom Fachmodul an den Intermediär VSDM kann ohne Pipelining über mehrere HTTP-connections erreicht werden.“ [VSDM-A_3066]

Strukturierte natürliche Sprache

Verbesserte Eindeutigkeit und Präzision durch (u.A. grammatikalische) Einschränkungen



Beispielsystem: Sophisten Satzschablonen

Quelle: Basiswissen Requirements Engineering (Pohl, Rupp)

Beispiel:

VSDM-A_3066: "(Das Fachmodul VSDM) DARF NICHT (Pipelining [...]) verwenden."

```
PROCESS Ground_level_estimation;  
IF (O.OOGROUND EQ $TRUE) THEN  
  G.ZGROUND = G.ZOWN;  
ELSEIF (G.RADAROUT GT P.RADARLOST) THEN  
  G.ZGROUND = -P.ZLARGE;  
ELSEIF (G.RADAROUT GT 0) THEN  
  ...  
END Ground_level_estimation;
```

"TCAS II requirements were defined by the Minimal Operational Performance Standard (MOPS) document. The MOPS was expressed using a combination of English and approximately 7,000 lines of lowlevel pseudocode."

Mats P.E.Heimdahl and Nancy G.Leveson and Jon D. Reese. **Experiences From Specifying the TCAS II Requirements Using RSML.** 17th Digital Avionics Systems Conference, 1998.

Modellierung

Unified Modeling Language (UML)

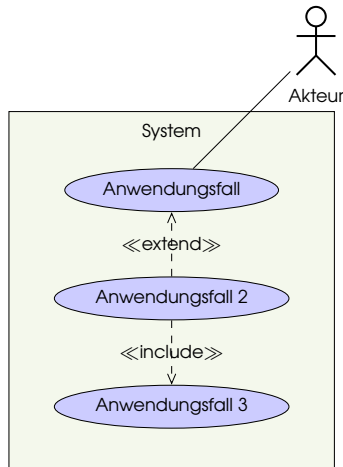
- Modellierung, Dokumentation, Spezifizierung und Visualisierung von komplexen Softwaresystemen.
- Standard der Object Management Group (OMG).
- Unterschiedliche Modellierungskonzepte auf einheitlicher Basis: 14 Diagrammtypen in Version 2.4.1
- Modell vs. Diagramm: UML-Modell besteht aus einem oder mehreren Diagrammen. Ein Diagramm entspricht einer bestimmten Sicht ("View") auf Modell (vgl. IEEE 1471).

Relevante Diagramme

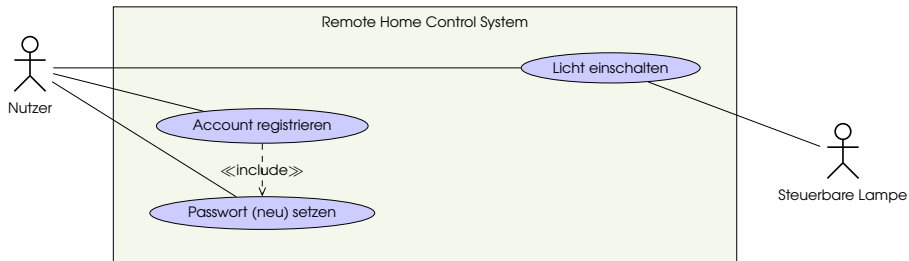
- **Anwendungsfalldiagramm**
High-Level Beschreibung der verschiedenen Anwendungsfälle und deren Beziehungen untereinander sowie zu Nutzen und anderen Systemen.
- **Klassendiagramm**
Struktur des Problembereichs (SWT / SoPra)
- **Aktivitätsdiagramm**
Darstellung von Abläufen von Aktivitäten, z.B. detaillierte Darstellung eines Anwendungsfalls
- **Zustandsdiagramm**
Ereignis-basierte Spezifikation von Reaktion des Systems auf Umgebung
(→ später in der Vorlesung)

UML Anwendungsfalldiagramme: Syntax

- Akteure: (System kann auch Akteur sein)
- Anwendungsfälle: Ellipsen mit Titel.
- Assoziationen: Linien zwischen Akteuren und Anwendungsfällen.
- Systemgrenzen: Rechtecke.
- include-Beziehungen: mit «include» gekennzeichnete gestrichelte Linie. Pfeil zum inkludierten Anwendungsfall
Inkludierter für den aufrufenden Anwendungsfall notwendig.
- extend-Beziehungen mit «extend» gekennzeichnete gestrichelter Linie. Pfeil vom erweiternden Anwendungsfall. Erweiternder kann vom aufrufenden Anwendungsfall aktiviert werden.



Beispiel: Anwendungsfalldiagramm



Mischformen: Schablone für Anwendungsfälle

In der Praxis findet man oft einen Mix verschiedener Dokumentationsmethoden

Beispiel:

- Top-Level: Anwendungsfall
- Zu jedem Anwendungsfall eine ausgefüllte Schablone mit Beschreibung des Anwendungsfalls und Varianten
- Unterstützung durch weitere UML Diagramme für mögliche Abläufe
- Referenz von relevanten Detail-Anforderungen, Qualitätsanforderungen oder UI-Anforderungen

Abschnitt	Inhalt
Bezeichner	UC-1
Name	Gerät schalten
Autoren	F. Howar
Priorität	...
Quelle	Nutzerbefragung
Verantwortlich	...
Kurz-Beschreibung	Der Nutzer schaltet ein Gerät im Haus von unterwegs ein.
Auslöser	Der Wunsch, ein Gerät zu schalten.
Vorbedingung	App installiert und Gerät registriert.
Nachbedingung	
Ergebnis	
Hauptscenario	App öffnen, Gerät aus Übersicht wählen, Gerät schalten
Alternativ-Szenarien	...
Ausnahme-Szenarien	...
Einzelne Anforderungen	REQ-F-10, ...
Qualitäten	REQ-Q-1, REQ-Q-2, ...

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Prüfbare Anforderungen
7. Literatur und Referenzen

Herausforderungen

■ Genauigkeit / Eindeutigkeit

- Kunden wissen nicht was sie wirklich wollen
- Kunden benutzen ihre eigene Fachsprache

■ Gemeinsame Erwartungen

- Verschiedene Beteiligte können widersprüchliche Anforderungen haben

■ Veränderung

- Anforderungen ändern sich während der Analyse und Entwicklung
- Neue Beteiligte mischen sich ein

Qualität von Anforderungen

Individuelle Anforderungen:

- Korrektheit:
Die Anforderung drückt die Erwartung der Betroffenen aus.
- Umsetzbarkeit:
Anforderung ist erfüllbar.
- Nachverfolgbarkeit:
Der Ursprung der Anforderung und ihre Entwicklung lassen sich feststellen.
- Eindeutigkeit:
Die Formulierung der Anforderung erlaubt nur eine gültige Interpretation.
- Prüfbarkeit:
Es lassen sich Kriterien oder Tests angeben, um nachzuweisen, ob die Anforderung erfüllt ist oder nicht.

Qualität von Anforderungen

Mengen von Anforderungen:

- Minimalität:
Es gibt keine doppelten oder irrelevanten Anforderungen.
- Konsistenz:
Die einzelnen Anforderungen widersprechen sich nicht.
- Vollständigkeit:
Die Anforderungen beschreiben das gewünschte System in allen relevanten Aspekten.
- Erweiterbarkeit / Anpassbarkeit:
Einzelne Anforderungen lassen sich unabhängig ändern.

Anforderungen validieren

Fragen

- Wird der Bedarf des Kunden vollständig abgedeckt?
- Verständlich formuliert?
- Konsistent mit anderen Anforderungen?
- Anforderung prüfbar?
- Änderbar ohne Einfluss auf andere Anforderungen?
- Realistisch mit Budget und Technologie?

Methoden

- Reviews
- Prototypen
- Erzeugung von Testfällen

Überprüfbarkeit von Qualitätsanforderungen

- Beschreibung von Qualitätsanforderungen oft unsystematisch und ungenau.
- Darauf achten, messbare Kriterien zu formulieren!

Eigenschaft	Messbare Formulierung
Geschwindigkeit	Ausgeführte Transaktionen / Sekunde Antwortzeit
Speicher	Mbytes Anzahl ROM
Benutzbarkeit	Einarbeitungszeit Umfang der Hilfe
Zuverlässigkeit	Durchschnittliche Zeit bis zu einem Absturz Wahrscheinlichkeit eines Ausfalls Fehlerhäufigkeit Verfügbarkeit
Robustheit	Zeit bis Wiederherstellung nach Ausfall Anteil fehlerhafter Vorgänge

Gruppierung als Basis der Qualitätssicherung

Bei mehreren tausend Anforderungen wird Struktur benötigt.

Sichten:

- Ausschnitte aus dem Anforderungsdokument für bestimmte Rollen (Entwickler, Kunde, etc.) erzeugen ("Sichten")

Kategorisieren:

- Art der Anforderung: Systemfunktionalität, Benutzerschnittstelle, Datenbank, Kommunikation, Qualität (z.B. Performanz)

Dann jeweils:

- Zusammengehörigkeit und Überlappungen von Anforderungen in einer großen Menge erkennen (z.B. von unterschiedlichen Autoren)
- Vollständigkeit der Anforderungen prüfen und
- Fehlende Anforderungen erkennen

Priorisierung von Anforderungen

Wofür?

- Grundlage von technischen und Management-Entscheidungen
- Kompromisse zwischen in Konflikt stehenden Anforderungen finden
- Planung von Releases (zuerst die wichtigen Anforderungen)
- Zertifizierung in kontrollierten Domänen (z.B. Automotive)

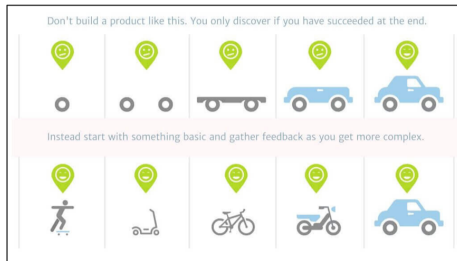
Beispiel

„**MUSS** bedeutet, dass es sich um eine absolut gültige und normative Festlegung bzw. Anforderung handelt.“

„**SOLL** beschreibt eine dringende Empfehlung. Abweichungen zu diesen Festlegungen sind in begründeten Fällen möglich. Wird die Anforderung nicht umgesetzt, müssen die Folgen analysiert und abgewogen werden.“

„**KANN** bedeutet, dass die Eigenschaften fakultativ oder optional sind. Diese Festlegungen haben keinen Normierungs- und keinen allgemeingültigen Empfehlungscharakter!“

Priorisierung: Minimal Viable Product



Minimal Viable Product = Kleinstes mögliches Produkt

- Schnell im Markt
- Schnell Feedback von Stakeholdern
- Kleineres Risiko
- Basis für Vertragsgestaltung

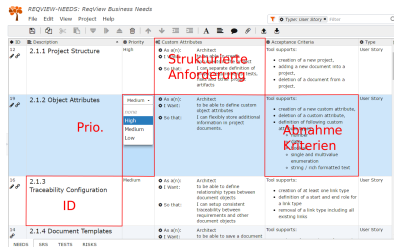
<https://thinkmobiles.com/blog/what-minimum-viable-product/>

Anforderungen verwalten

Was verwalten:

- Anforderung
- Woher Anforderung kam (Autor, Betroffene)
 - Für wen ist sie wichtig
 - Vermeidet Phantome
 - Erlaubt gezielte Rückfragen
- Priorität
- Relation zu anderen Anforderungen
- Abnahmekriterien

Werkzeuge



- IBM Rational Doors
- PTC Integrity
- Vector PREEVision
- MS Excel
- ...

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
- 6. Prüfbare Anforderungen**
7. Literatur und Referenzen

Fragen

Motivation.

- Es kann hilfreich sein, Anforderungen formal zu spezifizieren.

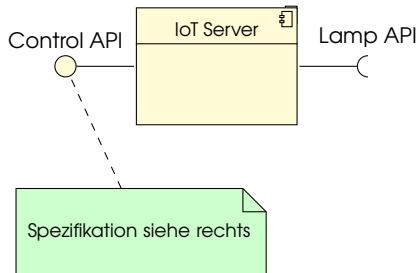
Vorgehen / Methoden.

- Wie beschreiben wir formal, was eine Komponente leisten soll?

Mögliche Analysen.

- Ist Anforderung erfüllbar?
- Sind Anforderungen widersprüchlich?

Beispiel: Schnittstelle an Komponente



Control API:

- `listLamps() → List<Lamp>`
- `getStatus(Lamp) → [on:Bool,err:Int]`
- `turnOn(Lamp) → [on:Bool,err:Int]`
- `turnOff(Lamp) → [on:Bool,err:Int]`

Anforderung: nach `turnOn(Lamp)` gilt $(on \vee (err > 0))$

Variablen / Typen

Typen:

- `Nat`: Ganze Zahlen
- `Boolean`: Boolesche Werte $\{0, 1\}$
- Aufzählungen. Z.B. Typ `State` mit Werten $\{\text{on}, \text{off}\}$

Variablen:

- Menge X von getypten Variablen

Getypete Ausdrücke

Für einen fixen Aufzählungstyp, z.B. **State**:

$e \in \mathbf{EExp}$ (Aufzählungs-Ausdrücke)

$e ::= z \mid c$

$z \in \mathbf{Var}$ Variable vom Aufzählungstyp **State**

$c \in \mathbf{State}$ konkreter Wert vom passenden Typ

Getypte Ausdrücke

$a \in \mathbf{AExp}$ (Arithmetische Ausdrücke)

$a ::= x \mid \mathbf{n} \mid (a_1 \text{ } op_a \text{ } a_2)$

$b \in \mathbf{BExp}$ (Boolesche Ausdrücke)

$b ::= y \mid \mathbf{true} \mid \mathbf{false} \mid \neg b \mid (b_1 \text{ } op_b \text{ } b_2) \mid (a_1 \text{ } op_r \text{ } a_2) \mid (z = c)$

$x \in X$ Variable Typ $\mathbf{Nat}$

$y \in X$ Variable Typ $\mathbf{Boolean}$

$z \in X$ Variable Typ $\mathbf{State}$

$\mathbf{n}$ Literal für $n \in \mathbb{Z}$

$c \in \mathbf{State}$

$op_a \in \mathbf{Op}_a$ arithmetische Op.: $+$, $-$

$op_b \in \mathbf{Op}_b$ Boolesche Op.: $\wedge$, $\vee$

$op_r \in \mathbf{Op}_r$ Relationale Op.: $>$, $<$, $=$

Beispiel

Variablen:

- $X = \{x_1, x_2\}$

Typen:

- x_1 von Typ `Nat`
- x_2 von Typ `State` mit Werten `{on, off}`

Ausdruck:

$$((x_2 = \text{off}) \wedge (x_1 > 50))$$

Belegungen

Belegung

Eine Belegung v von X ist eine Typ-konsistente Zuweisung von Werten zu Variablen in X . v ist eine Funktion und für $x \in X$ ist $v(x)$ der Wert mit dem x in v belegt ist.

- Typ-konsistent: $v(x)$ gehört zu Typ von x .
- Z.B., $v(x) \in \mathbb{Z}$ wenn x von Typ Nat .

Sei V_X die Menge aller möglichen typ-konsistenten Belegungen von X .

Beispiel

Variablen:

- $X = \{x_1, x_2\}$

Typen:

- x_1 von Typ Nat
- x_2 von Typ State mit Werten {on, off}

Belegung:

- v_1 mit $v_1(x_1) = 101$ und $v_1(x_2) = \text{on}$

Auswertung von Ausdrücken

- Set von Variablen X
- Ausdrücke aus **Exp** über Variablen aus X
- Fixer Aufzählungstyp z.B., **State** mit Werten $\{\text{on}, \text{off}\}$

Auswertung:

- Funktion $\alpha_X : \mathbf{AExp} \times V_X \mapsto \mathbb{Z}$
- Funktion $\beta_X : \mathbf{BExp} \times V_X \mapsto \{0, 1\}$
- Funktion $\gamma_X : \mathbf{Exp} \times V_X \mapsto \mathbf{State}$
- Belegung v von X

Auswertung von Aufzählungsausdrücken

Atome:

- $\gamma_X[\mathbf{on}, v] = \mathbf{on}$
- $\gamma_X[\mathbf{off}, v] = \mathbf{off}$
- $\gamma_X[z, v] = v(z)$

Auswertung von Booleschen Ausdrücken

Atome:

- $\beta_X[\mathbf{true}, v] = 1$
- $\beta_X[\mathbf{false}, v] = 0$
- $\beta_X[y, v] = v(y)$

Negation:

- $\beta_X[(\neg b), v] = 1$ gdw. $\beta_X[b, v] = 0$ (sonst 0)

Komposition:

- $\beta_X[(b_1 \wedge b_2), v] = 1$ gdw. $\beta_X[b_1, v] = 1$ und $\beta_X[b_2, v] = 1$ (sonst 0)
- $\beta_X[(b_1 \vee b_2), v] = 0$ gdw. $\beta_X[b_1, v] = 0$ und $\beta_X[b_2, v] = 0$ (sonst 1)

Auswertung von arithmetischen Ausdrücken

Atome:

- $\alpha_X[\mathbf{n}, v] = n$
- $\alpha_X[x, v] = v(x)$

Komposition:

- $\alpha_X[(a_1 + a_2), v] = \alpha_X[a_1, v] + \alpha_X[a_2, v]$
- $\alpha_X[(a_1 - a_2), v] = \alpha_X[a_1, v] - \alpha_X[a_2, v]$

Wir trennen hier nicht formal sauber zwischen Syntax und Semantik von $+$, $-$!

Boolesche Kombination:

Aufzählung:

- $\beta_X[(z = c), v] = 1$ gdw. $\gamma_X[z, v] = \gamma_X[c, v]$ (sonst 0)

Arithmetisch:

- $\beta_X[(a_1 \circ a_2), v] = 1$ gdw. $\alpha_X[a_1, v] \circ \alpha_X[a_2, v]$

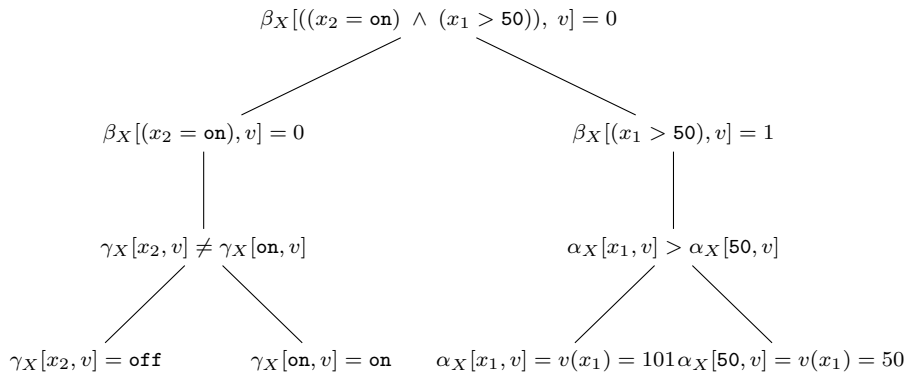
Für $\circ = \{<, =, >\}$

Beispiel

- Ausdruck: $((x_2 = \text{on}) \wedge (x_1 > 50))$
- Belegung: v mit $v(x_1) = 101$ und $v(x_2) = \text{off}$

Beispiel

- Ausdruck: $((x_2 = \text{on}) \wedge (x_1 > 50))$
- Belegung: v mit $v(x_1) = 101$ und $v(x_2) = \text{off}$



Umgang mit Variablen

Ausdrücke mit Variablen:

- Wir schreiben $\varphi[\mathbf{x}]$ um anzuzeigen, dass der Ausdruck φ über einer Menge $\mathbf{x}$ von Variablen formuliert ist

Beispiel:

- Für $\varphi = ((x_2 = \text{on}) \wedge (x_1 > 50))$
- $\varphi[\mathbf{x}]$ mit $\mathbf{x} = \{x_1, x_2\}$

Partielle Belegungen:

- Sei $\mathbf{z} \subset \mathbf{x}$ und $x \in \mathbf{x} \setminus \mathbf{z}$. Wir schreiben $v_{\mathbf{z}}[x \mapsto d]$ für die Belegung $v_{\mathbf{z} \cup \{x\}}$ mit
$$v_{\mathbf{z} \cup \{x\}}(y) = v_{\mathbf{z}}(y) \text{ für } y \neq x \text{ und}$$
$$v_{\mathbf{z} \cup \{x\}}(y) = d \text{ für } y = x$$
$$(\text{jeweils } y \in \mathbf{z} \cup \{x\}).$$

Erfüllbarkeit

Frage: Ist Ausdruck $\varphi[\mathbf{x}] \in \mathbf{BExp}$ erfüllbar

Existiert eine Belegung v von $\mathbf{x}$ so dass $\beta_{\mathbf{x}}[\varphi, v] = 1$?

Wir beschränken uns in SWK ab jetzt auf rein Boolesche Ausdrücke in konjunktiver Normalform (KNF)

$t \in \mathbf{BCNF}$ (rein Boolesche Ausdrücke in KNF)

$t ::= (c) \mid (\wedge (c))^*$

$c ::= l \mid (\vee l)^*$ (Clauses)

$l ::= y \mid \neg y$ (Literals)

$y \in X$ Variable Typ Boolean

$op_b \in \mathbf{Op_b}$ Boolesche Op.: $\wedge, \vee$

Wir verzichten für Lesbarkeit auf vollständige Klammerung

Beispiel: Erfüllbarkeit

Variablen:

- $X = \{x_1, x_2, x_3\}$ von Typ Nat

Ausdruck $(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (x_2 \vee \neg x_3)$ ist erfüllbar:

- z.B. v_1 mit $v_1(x_1) = 1$ und $v_1(x_2) = 0$ und $v_1(x_3) = 0$

Ausdruck $(x_1 \wedge \neg x_1)$ ist nicht erfüllbar.

Erfüllbarkeit und Widerspruch von Anforderungen

Sei $\varphi[\mathbf{x}], \psi[\mathbf{x}] \in \mathbf{BCNF}$.

■ Erfüllbarkeit (SAT):

φ ist erfüllbar, wenn Belegung $v_{\mathbf{x}}$ mit $\beta_{\mathbf{x}}[\varphi, v_{\mathbf{x}}] = 1$ existiert.

■ Widerspruch:

φ und ψ sind widersprüchlich, wenn $(\varphi \wedge \psi)$ nicht erfüllbar ist.

■ Implikation (Entailment):

φ impliziert ψ , wenn $(\varphi \wedge \neg\psi)$ nicht erfüllbar ist.

$(\varphi \wedge \neg\psi)$ ist nicht in KNF!

Beachte:

Wir verlangen, dass φ und ψ über den gleichen Variablen definiert sind!

DPLL-Algorithmus

- Rekursiver Algorithmus für die Entscheidung der Erfüllbarkeit von Booleschen Ausdrücken in KNF.
- DPLL: Davis, Putnam, Logemann, Loveland (1962)
- Worst-Case Kosten: $O(2^n)$ für Literalen über n Variablen (jede mögliche Belegung)
- Im Durchschnitt (in der Praxis) oft performant
- Basis für bessere Algorithmen für das Erfüllbarkeitsproblem

DPLL-Algorithmus (Beobachtungen)

Pure Literale

- Wenn im Ausdruck nur x vorkommt kann x mit 1 belegt werden
- Wenn im Ausdruck nur $\neg x$ vorkommt kann x mit 0 belegt werden

Unit Klauseln

Sei $\mathbf{z} \subset \mathbf{x}$ und $v_{\mathbf{z}}$, so dass in Klausel c **genau ein** Literal zu einer Variable aus $\mathbf{x} \setminus \mathbf{z}$ vorkommt

- Wenn in c Literal x vorkommt muss x mit 1 belegt werden
- Wenn in c Literal $\neg x$ vorkommt muss x mit 0 belegt werden

DPLL-Algorithmus

Input: Formula $\varphi \in \mathbf{BCNF}$ mit Klauselmenge C über Variablen $\mathbf{x}$

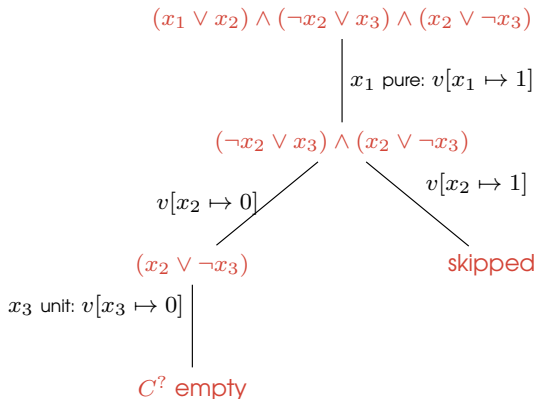
Output: *true* wenn φ erfüllbar ist, sonst *false*

```
1: procedure DPLL( $C, v_{\mathbf{z}}$  für  $\mathbf{z} \subseteq \mathbf{x}$ )
2:    $C^- \leftarrow \{c \in C : c[v_{\mathbf{z}}] \rightarrow \text{false}\}$ 
3:   if  $C^-$  is not empty then
4:     return false
5:   end if
6:    $C^? \leftarrow C \setminus \{c \in C : \text{true} \rightarrow c[v_{\mathbf{z}}]\}$ 
7:   if  $C^?$  is empty then
8:     return true
9:   end if
10:  if unit clause for literal  $x$  or  $\bar{x}$  in  $C^?$  then
11:    return DPLL( $C^?, v_{\mathbf{z}}[x \mapsto b]$ ) with  $b = 1$  for  $x$  and  $b = 0$  for  $\bar{x}$ 
12:  end if
13:  if pure literal  $x$  or  $\bar{x}$  in  $C^?$  then
14:    return DPLL( $C^?, v_{\mathbf{z}}[x \mapsto b]$ ) with  $b = 1$  for  $x$  and  $b = 0$  for  $\bar{x}$ 
15:  end if
16:  Pick  $x$  from  $\mathbf{x} \setminus \mathbf{z}$ 
17:  return DPLL( $C^?, v_{\mathbf{z}}[x \mapsto 0]$ )  $\vee$  DPLL( $C^?, v_{\mathbf{z}}[x \mapsto 1]$ )
18: end procedure
```


Beispiel: DPLL-Algorithmus

$$(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (x_2 \vee \neg x_3)$$

Beispiel: DPLL-Algorithmus



- Testet eine Belegung anstelle von $2^3 = 8!$

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Prüfbare Anforderungen
7. Literatur und Referenzen

Zusammenfassung

Anforderungen

- Erwartungen von Beteiligten / Betroffenen an ein System und Entwicklungsprozess
- Basis für spätere Phasen der Entwicklung
- Basis für Verträge über Software Entwicklung
- Erfolgsfaktor in Projekten

Typen von Anforderungen

- Funktionale Anforderungen
- Qualitätsanforderungen
- Randbedingungen

Qualität von Anforderungen

- Korrektheit, Eindeutigkeit, Prüfbarkeit, Umsetzbarkeit, Nachverfolgbarkeit
- Minimalität, Vollständigkeit, Konsistenz, Erweiterbarkeit / Anpassbarkeit

Zusammenfassung

Erheben und Management von Anforderungen

- Betroffene identifizieren
- Vier Schritte: Erheben, Dokumentieren, Abstimmen, Verwalten
- Verschiedene Methoden zum Erheben: Beobachtung, Befragung, Recherche, ...
- Validieren: Qualität, Konsistenz, Korrektheit sicherstellen mit Hilfe von Reviews, Prototypen, ...
- Abstimmen: Priorisieren, Gruppieren und Klassifizieren

Dokumentation von Anforderungen

- Wichtig: Überprüfbarkeit von Anforderungen
- Verschiedene Methoden (von natürlicher Sprache bis zu vollständig formal)
 - Natürliche Sprache
 - Satzschablonen
 - Schablonen für Anwendungsfall
 - Anwendungsfalldiagramme
 - Weitere UML Diagramme
 - Formale Logik

Was nicht besprochen wurde

- Details unterstützender Techniken
- Wie sind Anforderungsdokumente aufgebaut
- Details zu den Techniken zum Erheben von Anforderungen
- Techniken zum Präzisieren von Anforderungen
- Kosten / Nutzen Abschätzung für die Priorisierung von Anforderungen
- ...

Diese Themen werden in der genannten Literatur abgehandelt

- Pohl, Klaus, and Chris Rupp. Basiswissen Requirements Engineering : Aus- und Weiterbildung nach IREB-Standard zum Certified Professional for Requirements Engineering Foundation Level, dpunkt.verlag, 2015. ProQuest Ebook Central, Online lesbar [1]
- Sommerville, Ian. Software Engineering. 9. Aufl. Pearson, 2012. Kap. 4. isbn: 978-3868940992.
- Ebert, Christof. Systematisches Requirements Engineering. Anforderungen ermitteln, dokumentieren, analysieren und verwalten. 5. Aufl. dpunkt.verlag GmbH, 2014. isbn: 978-3-86490-139-3.

[1]: <http://ebookcentral.proquest.com/lib/dortmundtech/detail.action?docID=2029882>.

Standards

- **IEEE Std 610.12-1990** Institute of Electric and Electronic Engineers: IEEE Standard Glossary of Software Engineering Terminology (IEEE Std 610.12-1990). IEEE Computer Society, New York, 1990.
- **IEEE Std 830-1998** Institute of Electric and Electronic Engineers: IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). IEEE Computer Society, New York, 1998.
- **ISO/IEC 25010:2011** International Organisation for Standardization: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models, Genf 2011.
- **ISO/IEC/IEEE 29148:2011** International Organisation for Standardization: Systems and software engineering – Life cycle processes – Requirements engineering, Genf 2011.
- **OMG 2007** OMG: Unified Modeling Language: Superstructure, Version 2.1.1. OMG document formal/2007-02-05.
- **SEI 2006** Software Engineering Institute : CMMI for Development (CMMI-Dev), V1.2, Technical Report CMU/SEI-2006-TR-008 – ESC-TR-2006-008. Carnegie Mellon, Software Engineering Institute, Pittsburgh, PA 2006.